

CLAUDE CODE

EN VERIMLI GELISTIRME YAPISI

Model routing, context yonetimi, CLAUDE.md, Skills, Subagents, Hooks, Sandbox, Permissions ve buyuk is orkestrasyonu

Uygulama hedefi: Claude Code'a bu PDF'i verip mevcut repoyu analiz ettirerek gerekli ayarlari guvenli sekilde kurdurmak.

Onerilen varsayilan: SONNET 5 + OPUS 5 ADVISOR + HIGH EFFORT + SANDBOX + KATMANLI CONTEXT

Hazirlanma tarihi: 7 Eylul 2026
Kaynak ailesi: code.claude.com resmi dokumantasyonu

1. Yoneticici Ozeti

Sonuc: Her gelistirme isteginden once modeli elle secmek yerine Claude Code, gunluk isi Sonnet 5 ile yurutmeli; zor karar, tekrarlayan hata veya completion-check anlarında Opus 5 Advisor kullanmalidir. Advisor kullanilamiyorsa planlama/uygulama ayrimi icin opusplan fallback olarak kullanilabilir.

Kaynak: [S2], [S3]

Katman	Varsayilan	Neden
Ana model	Sonnet 5	Gunluk coding icin resmi alias; hiz/kalite dengesi.
Zor karar destegi	Opus 5 Advisor	Claude ne zaman danisacagina kendisi karar verir; ana akis bozulmaz.
Effort	high	Resmi dokumanda zeka/token dengesi icin varsayilan seviye.
Cok buyuk proje	Fable 5.1	Tek oturumu asan, uzun ve otonom isler icin.
Izole arastirma	Subagents	Ana contexti dosya okuma ve arastirma gürültüsünden korur.
Tekrarlanan prosedür	Skills	Sadece gerektiğinde yuklenir; CLAUDE.md sismez.
Kesin otomasyon	Hooks	Her seferinde calismasi gereken formatter/guard gibi kurallar.
Yetki ve guvenlik	Sandbox + permission rules	Daha az onay penceresi, daha kontrollu otonomi.

Gunluk kullanım hedefi: `claude` ac -> “surada hata var, root cause bul, duzelt ve dogrula” de -> gerisini sistem yonetsin.

2. Model Routing Stratejisi

2.1 Varsayilan: Sonnet 5 + Opus 5 Advisor

Claude Code resmi model alias tablosunda `sonnet` gunluk coding, `opus` kompleks reasoning ve `fable` en zor/uzun gorevler icin konumlandiriliyor. Advisor araci ise ana modele ikinci ve genellikle daha guclu bir modeli kritik anlarda danisma imkani veriyor.

Kaynak: [S2], [S3]

Main session: sonnet
Advisor: opus
Effort: high

Advisor su an deneysel bir ozelliktir ve resmi dokumana gore Anthropic API gerektirir. Bedrock, Claude Platform on AWS, Google Cloud Agent Platform ve Microsoft Foundry uzerinde kullanilamaz. Bu nedenle kurulum, ortam yetenegini once tespit etmelidir.

2.2 Fallback: opusplan

Advisor yoksa veya kullanılmak istenmiyorsa `opusplan`, Plan Mode sirasinda Opus; execution sirasinda Sonnet kullanir. Bu hibrit model planlama sinirinda otomatik gecis yapar; Advisor ise gorevin ortasinda Claude karar verdiginde ikinci modele danisir.

Kaynak: [S2]

2.3 Fable ne zaman?

Fable 5.1 global default olmamali. Repo-wide migration, uzun root-cause investigation, buyuk mimari donusum ve tek bir oturumu asabilecek otonom islerde bilincli olarak secilmelidir. Resmi dokuman Fable modellerini “tasks larger than a single sitting” icin konumlandirir.

Kaynak: [S2]

Istek tipi	Model/Mod
Kucuk bug, normal feature, refactor, test	Sonnet 5 + Advisor
Belirsiz mimari karar / inatci hata	Sonnet 5 + Opus Advisor
Planlamasi kritik orta-buyuk is	opusplan (Advisor yoksa)
Tek seferlik cok zor reasoning	Prompt icinde `ultrathink`
Codebase-wide buyuk proje	Fable 5.1
500+ dosya migration/audit	Dynamic workflow / gerekirse ultracode

Kaynak: [S2], [S3], [S10]

3. Global Kullanici Ayarlari

Kisisel ve tum projelere uygulanacak model/effort/sandbox tercihleri `~/.claude/settings.json` kapsaminda tutulmalidir. Takimla paylasilacak proje kurallari ise repodaki `~/.claude/settings.json` dosyasina konur. Claude Code resmi dokumani bu kapsam ayrimini acikca tanimlar.

Kaynak: [S11]

Onerilen baslangic konfigurasyonu (Claude bunu mevcut ayarlarla merge etmeli, dosyayi korlemesine ezmemelidir):

```
{
  "$schema": "https://json.schemastore.org/claude-code-settings.json",
  "model": "sonnet",
  "advisorModel": "opus",
  "effortLevel": "high",
  "sandbox": {
    "enabled": true
  },
  "permissions": {
    "defaultMode": "auto",
    "deny": [
```

```

"Read(/.env)",
"Read(/.env.*)",
"Read(/secrets/**)"
],
"ask": [
  "Bash(git push *)",
  "Bash(gh pr create *)"
]
}
}

```

Uygulama notu: `permissions.defaultMode = auto` yalnızca hesabin/surfacenin Auto Mode desteği varsa etkinleştirilmeli. Destek yoksa mevcut varsayılan permission modu korunmalı ve güvenli allow/ask/deny kuralları kullanılmalı. `bypassPermissions` varsayılan olarak kullanılmamalı.

Kaynak: [S6], [S11]

Sandbox macOS, Linux ve WSL2 üzerinde desteklenir; native Windows desteklenmez. Native Windows ortamında Claude Code WSL2 içinde çalıştırılmalıdır. Sandbox global olarak `sandbox.enabled=true` ile etkinleştirilebilir.

Kaynak: [S5]

4. CLAUDE.md: Kisa, Katmanli, Kalici Bilgi

Her Claude Code session fresh context ile baslar. Kalici proje talimatları CLAUDE.md; Claude'un kendisinin öğrendiği patternler ise auto memory ile tasınir. CLAUDE.md her session basında yüklendiği için yalnızca her zaman gerekli, koddan kolayca çıkarılamayacak bilgiler bulunmalıdır.

Kaynak: [S4]

CLAUDE.md icine koy	CLAUDE.md icine koyma
Build/test/lint/typecheck komutları	Koddan okunabilecek dosya-dosya açıklamalar
Projeye özel mimari kararlar	Uzun API dokümantasyonu
Takim workflow kuralları	Standart dil best-practice dersleri
Non-obvious gotcha ve conventionlar	Sadece bazen gereken prosedürler
Definition of Done / verification kuralı	Her şeyi "IMPORTANT" yapma

Kaynak: [S1], [S4]

Onerilen repo yapisi:

```

repository/
|-- CLAUDE.md
|-- .claude/
|   |-- settings.json
|   |-- rules/
|       |-- frontend.md
|       |-- backend.md
|       |-- testing.md

```

```

| |-- skills/
| | |-- bugfix/SKILL.md
| | |-- feature/SKILL.md
| | |-- migration/SKILL.md
| | |-- verify/SKILL.md
| | `-- review/SKILL.md
| `-- agents/
|     |-- researcher.md
|     |-- code-reviewer.md
|     `-- test-verifier.md
|-- frontend/CLAUDE.md    # sadece gerekliyse
`-- backend/CLAUDE.md    # sadece gerekliyse

```

Alt dizinlerdeki CLAUDE.md dosyaları ilgili dizin okunmaya başlandığında yüklenebilir; `.claude/rules/` ile de path-scoped kurallar tanımlanabilir. Büyük monorepolarda bu, context israfını azaltır.

Kaynak: [S4]

5. Skills, Subagents ve Hooks - Hangisi Ne İçin?

Arac	Kullan	Kullanma
CLAUDE.md	Her session geçerli kısa kurallar	Uzun prosedür deposu
Skill	Tekrarlanan/on-demand workflow	Her zaman yüklenmesi gereken basit kural
Subagent	Çok dosya okuyan araştırma, review, verification	Tek satırlık basit iş
Hook	Sıfır istisna ile çalışması gereken otomasyon	Esnek karar gerektiren davranış
MCP	Structured external system access	Zaten iyi bir CLI varsa sırf entegrasyon olsun diye

Kaynak: [S1], [S7], [S8], [S9]

5.1 Minimum baslangic Skills

- `bugfix`: reproduce -> root cause -> smallest correct fix -> regression test -> verify
- `feature`: inspect existing patterns -> plan only if needed -> implement -> test -> verify
- `migration`: inventory -> batch safely -> verify deprecated usage is zero
- `verify`: run the repo-specific relevant tests/build/lint/typecheck, report evidence
- `review`: correctness/requirements focused review; style preference ile gereksiz over-engineering yapma

Skills `.claude/skills/<name>/SKILL.md` altında tutulur ve Claude uygun olduğunda otomatik uygulayabilir veya `/skill-name` ile çağrılabilir.

Kaynak: [S7]

5.2 Minimum baslangic Subagents

- `researcher`: geniş dosya taramasını ana context dışında yapar.
- `code-reviewer`: diffi fresh context ile correctness/edge-case açısından denetler.
- `test-verifier`: değişikliğe uygun test/build/typecheck kanitını kontrol eder.

Subagent'lar kendi context window'larında çalışır. Resmi best-practice rehberi, araştırma için subagent kullanmayı ana contexti temiz tutmanın temel yöntemlerinden biri olarak önerir.

Kaynak: [S1], [S8]

6. Verification Loop: “Bitti” Demeden Kanıt

Claude Code için en yüksek kalıdracı uygulama, ajana sonucunu doğrulayabileceği bir test/build/lint/screenshot sinyali vermektir. Resmi rehber, verification olmayan durumda kullanıcının doğrulama dongusune donustugunu vurgular.

Kaynak: [S1]

Her task için minimum davranış:

1. Sorunu / gereksinimi anla.
2. Gerekliyse mevcut patternleri ve root cause'u incele.
3. En küçük doğru değişiklik yap.
4. İlgili testleri ekle/guncelle.
5. Uygun test + build/typecheck/lint çalıştır.
6. Sonucu iddia etmek yerine kanıtla (komut + sonuc).
7. Kritik/büyük işte fresh-context reviewer ile ikinci kontrol yap.

Büyük ve unattended işlerde `/goal` veya deterministik Stop hook kullanılabilir. Review için fresh-context subagent, implement eden agentin kendi işini notlamasından daha sağlam ikinci görüş sunar.

Kaynak: [S1]

7. Session Yönetimi

Durum	Davranış
Yeni/unrelated task	Yeni session veya `/clear`
Aynı task ertesi gün	`--resume` / `--continue`
Context sisiyor	`/compact <ne korunacak>`
Yan soru contextte kalmasin	`/btw`
İki kez aynı konuda düzeltme yaptın	`/clear` + daha iyi ilk prompt
Paralel iki Claude aynı repo	Her biri ayrı worktree

Kaynak: [S1], [S12]

Mevcut “her bug/feature için ayrı session aç” alışkanlığı doğrudur. Resmi dokümanda context temel kısıt olarak tanımlanır; unrelated işlerin aynı sessionda birikmesi yaygın failure patternidir.

Kaynak: [S1]

8. Büyük İşler: Worktrees, Dynamic Workflows, Fable

Aynı repository üzerinde paralel session çalışacaksa `claude --worktree <name>` veya `-w` ile izole git worktree kullanılmalıdır. Böylece bir session'ın editleri diğerinin dosyalarına dokunmaz.

```
claude --worktree feature-auth  
claude --worktree bug-cache
```

Dynamic workflows; codebase-wide audit, 500 dosyalik migration ve cok sayida cross-check edilen arastirma gibi onlarca-yuzlerce subagent gerektiren isler icindir. Gunluk feature gelistirmede varsayilan olmamalidir.

Kaynak: [S10]

`ultracode`, resmi model ayarlarında xhigh reasoning ile substantive tasklar icin dynamic workflow orkestrasyonu acan Claude Code ayaridir. Gunluk default yapmak yerine gereken buyuk islerde bilincli kullanmak daha dengelidir.

Kaynak: [S2]

9. CLAUDE CODE'A VERILECEK UYGULAMA SPESIFIKASYONU

Bu bolum PDF'i Claude Code'a verdiginizde dogrudan uygulanacak talimattir.

ROL: Bu repository icin Claude Code developer-experience ve agentic-engineering setup uzmanisin.

AMAÇ:

Bu PDF'teki resmi-doc tabanlı hedef yapisini mevcut repository'ye uygula. Repo hangi teknoloji stack'ini kullaniyorsa otomatik tespit et; React/.NET/PHP/Java vb. varsayma.

DEGISIKLIK ONCESI ZORUNLU ANALIZ:

1. `claude --version` ve ilgili feature availability'yi kontrol et.
2. Mevcut `~/claude/settings.json`, `claude/settings.json`, `claude/settings.local.json`, `CLAUDE.md`, `CLAUDE.local.md`, `claude/rules/`, `claude/skills/`, `claude/agents/`, hooks ve MCP configlerini incele.
3. Mevcut faydali ayarlari KORU. Dosyalari korlemesine overwrite etme; merge et.
4. Repo'nun build/test/lint/typecheck komutlarini package/config/CI dosyalarından tespit et.
5. Mimari sinirlari, workspace/monorepo yapisini ve mevcut conventionlari analiz et.

MODEL STRATEJISI:

- Anthropic API + Advisor destekleniyorsa: main=`sonnet`, advisor=`opus`, effort=`high`.
- Advisor desteklenmiyorsa: main=`sonnet`; buyuk/planlamasi kritik tasklarda `opusplan` kullanimi icin dokumantasyon/skill ekle.
- Fable'i global default yapma. Sadece codebase-wide / long-running engineering projectlerinde bilincli kullanim icin rehber ekle.
- `ultracode` ve dynamic workflow'u gunluk default yapma.

GLOBAL KULLANICI AYARLARI:

- Sandbox platformda destekleniyorsa global etkinlestir.
- Auto permission mode hesap/surface tarafında destekleniyorsa kullan; degilse mevcut modu koru.
- `.env`, `.env.\*` ve secrets klasorlerini read deny ile koru; mevcut projenin secret yapisini tespit edip guvenli deny listesi genislet.
- `git push`, PR creation, destructive git/database/cloud/infrastructure islemlerini otomatiklestirmeden once uygun ask/deny sinirlari koy.
- `bypassPermissions` varsayilan yapma.

PROJECT CONTEXT:

- Root CLAUDE.md'yi kısa tut. Sadece koddan cikarilamayacak build/test komutlari, architecture kararlar, workflow, gotcha ve Definition of Done bulunsun.
- Uzun/sadece bazen gereken bilgileri Skills veya path-scoped `claude/rules/` dosyalarina tası.
- Buyuk monorepoda alt CLAUDE.md sadece gercekten domain-specific talimat gerekiyorsa olustur.

SKILLS:

Repo'ya uyarlayarak minimum su workflow'lari kur:

- bugfix
- feature
- migration
- verify
- review

Her skill icinde repo'ya ozel komutlari ve verification criteria kullan. Generic bos boilerplate yazma.

SUBAGENTS:

Gercek fayda varsa su rolleri kur:

- researcher: read-heavy investigation, main contexti korur
- code-reviewer: fresh-context correctness/edge-case review
- test-verifier: degisikligi dogrulayan test/build/lint/typecheck evidence

Subagent modelini goreve gore sec; pahali modeli mekanik islere zorunlu kilma.

HOOKS:

Yalnizca deterministik ve sifir-istisna gerektiren islemler icin ekle (ornegin formatter).

Her Edit sonrasinda full build/full test gibi agir hooks ekleme.

Permission ile cozulabilecek seyi hook ile cozmeye calisma.

MCP / CLI:

Mevcut CLI (gh, docker, dotnet, npm/pnpm, etc.) yeterliyse sirf entegrasyon icin MCP ekleme. Structured external access gercekten gerekiyorsa MCP oner veya kur.

VERIFICATION:

- Degisikliklerden sonra `claude doctor`, `/status` veya uygun CLI kontrolleriyle settings'in yuklendigini dogrula.
- JSON dosyalarini schema/parse acisindan dogrula.
- CLAUDE.md'nin `/context` tarafinda yuklendigini dogrula.
- Repo icin ilgili test/lint/typecheck/build komutlarini calistir.
- Olusturdugun skills/agents dosyalarinin syntax ve discovery'sini kontrol et.
- Son raporda: degisen dosyalar, nedenleri, aktif model/advisor/effort/sandbox/permission durumu, hangi feature'in ortam nedeniyle uygulanamadigi ve manuel olarak ne zaman Fable/opusplan/ultracode kullanilacagi yazsin.

KISIT:

Bu kurulumu repository'nin mevcut ihtiyaclarina gore MINIMAL ve bakimi kolay tut. PDF'te adi gecen her ozelligi sirf var diye kurma. Resmi Claude Code dokumani ve mevcut CLI surumu PDF'teki bir ayrintıyla celisirse, GUNCEL RESMI DOKUMAN/CLI DAVRANISI ustundur ve farki final raporda belirt.

10. Kabul Kriterleri

- Yeni normal session Sonnet ile basliyor; kullanıcı her prompt once model secmiyor.
- Advisor kullanilabilen ortamda Opus advisor persistent olarak aktif; kullanilamiyorsa fallback strateji dokumante.
- High effort aktif veya modelin resmi default high davranisi korunuyor.
- Sandbox desteklenen platformda aktif ve settings yuklenmis.
- Secret dosyolari okunamiyor; destructive islemler serbest degil.
- CLAUDE.md kısa ve repo-spesifik; uzun prosedurler Skills/Rules altinda.
- En az bugfix/feature/verify workflowlari repo komutlariyla gercekten calisabilir.
- Research/review ihtiyaci varsa subagentlar main contexti koruyacak sekilde tanimli.
- Her taskin "done" tanimi verification evidence iceriyor.
- Paralel session icin worktree rehberi mevcut.
- Claude doctor/status/context kontrolleri hata vermiyor.
- Mevcut ayarlar kaybolmadi; yapilan degisiklikler final raporda listelendi.

11. Resmi Kaynaklar

Bu rapor 7 Eylul 2026 tarihinde asagidaki resmi Claude Code dokumanlari ile dogrulanmistir. Claude Code hizla gelistigi icin kurulum sirasinda mevcut CLI surumu ve resmi dokuman her zaman son otoritedir.

[S1] Best practices for Claude Code

<https://code.claude.com/docs/en/best-practices>

Verification loop, planlama, CLAUDE.md, permissions, skills, subagents, session/context yonetimi, parallel/review.

[S2] Model configuration

<https://code.claude.com/docs/en/model-config>

Sonnet/Opus/Fable/Haiku aliaslari, opusplan, effort, ultrathink, ultracode, Fable kullanim alani.

[S3] Advisor tool

<https://code.claude.com/docs/en/advisor>

Sonnet + Opus advisor modeli, deneysel durum, Anthropic API kosulu, accepted pairings.

[S4] How Claude remembers your project

<https://code.claude.com/docs/en/memory>

Fresh context, CLAUDE.md, auto memory, katmanli/project/local kurallar.

[S5] Sandboxing

<https://code.claude.com/docs/en/sandboxing>

Sandbox platform destegi, global enable, filesystem/network izolasyonu.

[S6] Permissions

<https://code.claude.com/docs/en/permissions>

Auto/default/plan/dontAsk/bypass modes, allow/ask/deny precedence ve rule syntax.

[S7] Skills

<https://code.claude.com/docs/en/skills>

`.claude/skills/.../SKILL.md`, on-demand reusable workflows.

[S8] Subagents

<https://code.claude.com/docs/en/sub-agents>

Custom subagent tanimi, kendi contexti/tools/modeli.

[S9] Hooks

<https://code.claude.com/docs/en/hooks>

Deterministik lifecycle otomasyonu ve hook configuration.

[S10] Dynamic workflows

<https://code.claude.com/docs/en/workflows>

Codebase-wide audit/migration, cok sayida subagent orkestrasyonu.

[S11] Settings

<https://code.claude.com/docs/en/settings>

User/project/local/managed scope, settings precedence, schema, verification.

[S12] Worktrees

<https://code.claude.com/docs/en/worktrees>

Paralel sessionlari izole git worktree ile calistirma.

[S13] Claude Code Overview

<https://code.claude.com/docs/en/overview>

Claude Code yetenekleri, CLAUDE.md, skills, hooks, agents, MCP genel cercevesi.

[S14] Claude Code Settings JSON Schema

<https://json.schemastore.org/claude-code-settings.json>

Settings dosyalari icin schema ve editor validation.

Kisa ezber: Sonnet yapar -> Opus Advisor kritik anda dusunur -> Subagents contexti korur -> Skills tekrar eden isi paketler -> Hooks zorunlu otomasyonu garanti eder -> Fable buyuk projeyi sahiplenir.